Technical Journal Entry: Resource Storage & Seed Bank

- **Module:** Resource Storage and Seed Bank Initialization (Plant class + UI unlock)
- **Time/Space Complexity:** O(1) for Plant object instantiation; O(k) time/space for populating the seed dropdown, where k is the number of plant tuples passed to `addPlantsToDropdown`.
- **Architectural Choice:** Used a simple class (`Plant`) to model garden resource entities with encapsulated state (`name`, `water_need`, `health`), then called two bridge functions (`updateResources`, `addPlantsToDropdown`) to sync Python state with the JS/DOM layer and unlock the Resources panel.
- **AI Reflection Note:**
    - **The Prompt Used (RTCC):**
      "Act as an expert Python developer. Task: Write a Plant class and initialize the garden's resource system and seed dropdown. Context: Python 3.11+, vanilla Python compatible with browser Pyodide execution, must call exact JS bridge functions `updateResources` and `addPlantsToDropdown`. Constraints: Must include a `__init__` storing name, water_need, and health=100, print a creation message, create at least one Plant object, call `updateResources` with exact specified values, call `addPlantsToDropdown` with at least 3 tuples, and end with an exact success print statement."
    - **What the AI Initially Got Wrong:**
      N/A — output matched requirements on first generation (no missing fields, correct function signatures, correct tuple format).
    - **How I Fixed It:**
      N/A — no correction cycle was needed for this module.
- **Screenshots:** *[insert screenshot of unlocked Resources panel + console output here]*